

Embedded Program Development

EMBEDDED — Lecture 3

Typical Program-generation Flow

- Compile -> Assemble -> Link -> Download.
- Generated executable file (or program image) stored in program memory (normally on-chip flash memory), to be fetched by the processor.

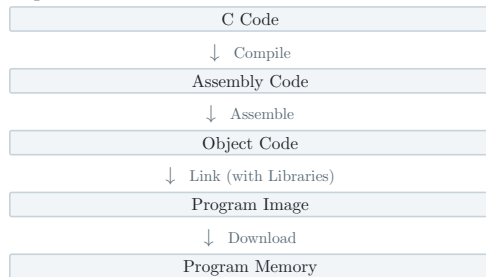


Fig. 1: Typical program-generation flow

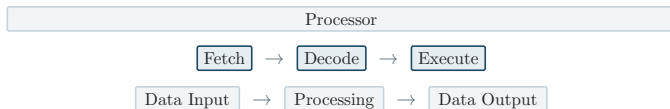


Fig. 2: Processor: Instruction Fetch, Decode, Execute; data processing

Program Image and Memory Sections

- The linker turns many object-file sections into one image whose bytes have defined storage and runtime addresses.

Cortex-M4 Memory Map

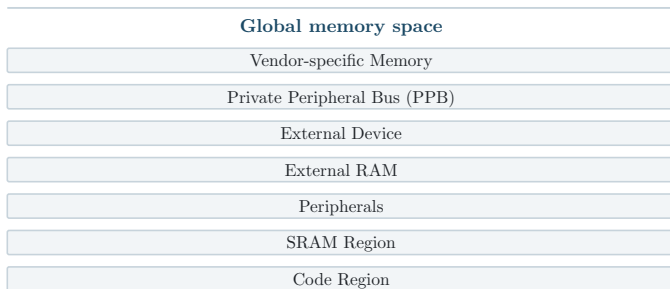


Fig. 3: Cortex-M4 memory map example: regions and their address ranges

Region	Address range
Code	0x00000000 – 0x1FFFFFFF
SRAM	0x20000000 – 0x3FFFFFFF
Peripherals	0x40000000 – 0x5FFFFFFF
External RAM	0x60000000 – 0x9FFFFFFF
External device	0xA0000000 – 0xDFFFFFFF
Private Peripheral Bus (PPB)	0xE0000000 – 0xE00FFFFFFF
Vendor-specific memory	0xE0100000 – 0xFFFFFFFF

On-chip blocks: Cortex-M4 core with PPB (SCS: NVIC, Debug CTRL), connected via AHB/APB to on-chip FLASH (code region), on-chip SRAM (SRAM region), Peripheral Region (Timer, UART, GPIO), External memory interface (external RAM region) and External device interface (external device region); external SRAM/FLASH, external LCD, SD card attached below.

Cortex-M4 Program Image

Program image (also called **executable file**) piece of fully integrated code that is ready to execute; contains machine instructions and constant data; addresses in the image match the target memory layout.

Program Image Contents

In the Cortex-M4, the program image includes:

- Vector table
- C start-up routine
- Program code
- C library code

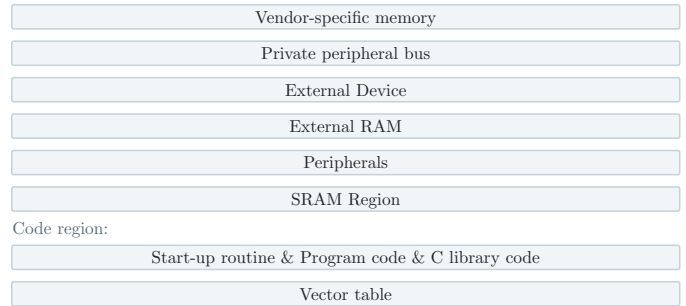


Fig. 4: Program image occupies the code region of the global memory space

Vector Table

- Contains the starting addresses of interrupt vectors:
 - Entry 0 stores the value of the main stack point (MSP)
 - Entry 1 stores the Reset Handler address
 - Later entries store exception and interrupt handler addresses

Address	Content
0x00000000	Initial MSP value
0x00000004	Reset vector
0x00000008	NMI vector
0x0000000C	Hard fault vector
0x00000010	Reserved
0x0000002C	SVCall
0x00000030	Reserved
0x00000038	PendSV
0x0000003C	SysTick
0x00000040	External Interrupts

C Start-up Code

- Used to set up data memory and the initialization of values for global data variables.
- Inserted by the compiler/linker automatically, e.g., labeled as `__start` by the GNU C compiler.

Program and Library Code

- Program code: instructions and data generated from the application program.
- C library code: object codes inserted into the program image by linkers.

Systems Initialization

After reset, the processor First reads the initial MSP value, then reads the reset vector, then branches to the start of the program execution address (reset handler), subsequently executes program instructions.

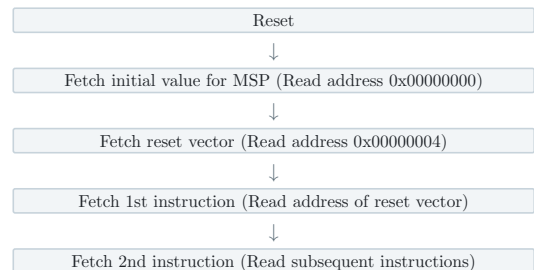


Fig. 5: Reset sequence in ARM Cortex processors

Program Image in Global Memory

- Program image stored in the code region in global memory:
 - up to 512 MB memory space range from 0x00000000 to 0x1FFFFFFF;
 - usually implemented on non-volatile memory, such as on-chip FLASH memory.
- Normally separated from program data, which is allocated in the SRAM region.

- Code region mainly used for program image (e.g., on-chip FLASH); SRAM region mainly used for data memory (e.g., on-chip SRAM, SDRAM).

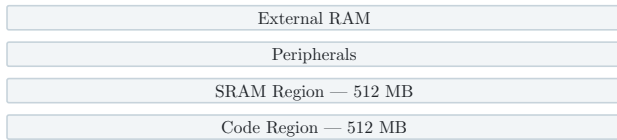


Fig. 6: Global memory space: code region and SRAM region

How Data Is Stored in SRAM

Typically the data can be divided into three sections: static data, stack and heap.

- Static data: global variables and static variables.
- Stack: local variables, parameter passing in function calls, registers saving during exceptions, etc.
- Heap: dynamically reserved by function calls, such as `alloc()` and `malloc()`.
 - Generally not preferred in embedded systems.



Fig. 7: SRAM layout: stack grows downwards, heap grows upwards, static data at low addresses

Memory Placement Begins with Mutability

Can the information change?

- No** Put it in read-only nonvolatile memory: instructions, constant strings, constant operands, initialization values
- Yes** Put it in volatile (read/write) memory: variables, intermediate computations, return address

```
int a, b;
const char c=123;
int d=31;
void main(void) {
    int e;
    char f[32];
    e = d + 7;
    a = e + 29999;
    strcpy(f, "Hello!");
}
```

Fig. 1: Example C program used throughout the placement examples

Data Lifetime

How long does the data need to exist? Reuse memory if possible.

- Statically allocated:
 - exists from program start to end;
 - each variable has its own fixed location;
 - space is not reused.
- Automatically allocated:
 - exists from function start to end;
 - space can be reused.
- Dynamically allocated:
 - explicit allocation to explicit de-allocation;
 - space can be reused.

Executable Image Sections

Executable image contains initialized data and uninitialized data sections:

- `.data` and `.sdata`: contain the initial values for the global and static variables.
- `.bss` and `.sbss`: uninitialized (or zero initialized) data sections (their content is empty).
- `.const`: constant data is part of this section, which is read-only.

Section	Contents	Storage / run-time
<code>.text</code>	Machine instructions	Flash → Flash
<code>.rodata</code> / <code>.const</code>	Read-only constants	Flash → Flash
<code>.data</code>	Initialized writable objects	Initial bytes in flash → SRAM
<code>.bss</code>	Zero-initialized writable objects	Size only → SRAM
Stack	Call frames and saved context	Not stored in image → SRAM
Heap	Dynamic allocations	Not stored in image → SRAM

Program Memory Use



Fig. 2: Program memory use: RAM sections and Flash ROM sections

Variables of the example map to sections as follows:

- `int a, b;` → Zero-initialized data (`.bss/.sbss`)
- `const char c=123;` → Constant data (`.const`)
- `int d=31;` → Initialization data (`.data/.sdata`)
- `int e;`, `char f[32];` → Stack
- `e = d + 7;`, `a = e + 29999;`, `strcpy(f, "Hello!");` → Program `.text`
- `"Hello!"` → Constant data (`.const`)

WHY? Zero-initialized data are filled with zeros and initialized data are copied from Flash ROM to RAM at start-up.

C Run-Time Start-Up Module

After reset, MCU must:

- Initialize hardware, ...
- Initialize C or C++ runtime environment:
 - set up heap memory, initialize variables.

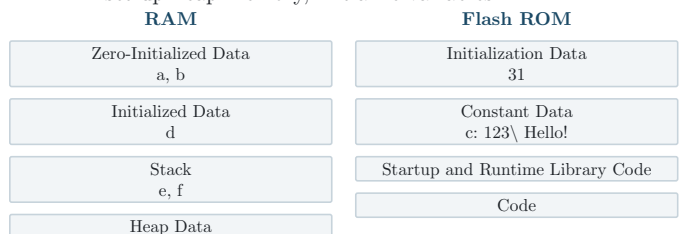


Fig. 8: Start-up: zero-initialized data filled with zeros, initialized data copied from Flash ROM

Loading Program Image

- Transferring an executable image built for a target embedded system from the host onto the target:
 - the entire executable image is burned into the ROM or flash memory using special equipment.
- Each time the code changes due to bugs or code addition:
 - reprogram the ROM or the flash memory using special equipment.
 - e.g., JTAG interface (JTAG — Joint Test Action Group).

An Example Booting Procedure

- After powered on:
 - fetch and execute code from a predefined and hard-wired address offset;
 - the code contained at this memory location is called the reset vector.
- The reset vector is usually a jump instruction into the real initialization code, called the bootstrap code.
 - The reason for jumping to another part of memory is to keep the reset vector small.

Booting Steps

Two CPU registers are of concern:

- Instruction Pointer (IP) register: points to the next instruction (code in the `.text` section).
- Stack Pointer (SP) register: points to the next free address in the stack.

The SP must be set appropriately at start-up; the stack is created from a space in RAM.

1. IP is hardwired to execute the first instruction in memory (the reset vector).
2. The reset vector jumps to the first instruction of the `.text` section in ROM.
3. The `.data` section of the boot image is copied into RAM (it is both readable and writable).
4. Space is reserved in RAM for the `.bss` section: nothing to transfer because the content for the `.bss` section is empty.
5. Stack space is reserved in RAM; the SP register is set to point to the beginning of the newly created stack.

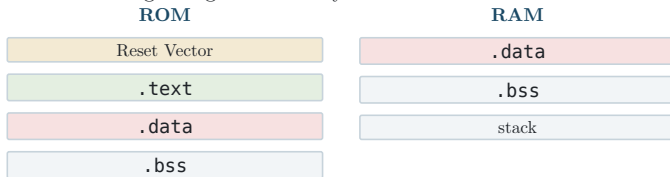


Fig. 3: Boot procedure: `.text` executes from ROM; `.data` copied to RAM; `.bss` and stack reserved in RAM

At this point, the boot completes: the CPU continues to execute the code in the `.text` section.

Linker Map File

- The linker creates a single executable image for the target embedded system.
 - Merge sections from the different object files into program segments.
- Linker commands (called linker directives) control how the linker combines the sections and allocates the segments into the target system.

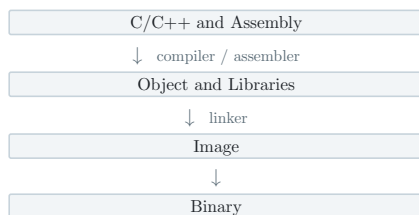


Fig. 9: Linker map file: from C/C++ and assembly sources to binary

Linker Command File

- The linker directives are kept in the linker command file.
 - Two of the more common directives supported by most linkers are `MEMORY` and `SECTION`.
- The `MEMORY` directive can be used to describe the target system's memory map.
 - An embedded developer needs to be familiar with the addressable physical memory on a target system.

MEMORY Directive

- The `MEMORY` directive defines:
 - the types of physical memory present in the target system;
 - the address range occupied by each physical memory block.
- Format:

```
MEMORY {
    area-name : org = start-address, len = number-of-bytes
    ...
}
```

Example:

```
MEMORY {
    ROM: origin = 0x0000h, length = 0x0020h
    FLASH: origin = 0x0040h, length = 0x1000h
    RAM: origin = 0x1000h, length = 0x10000h
}
```

Block	Start	End
ROM	0x00000	0x0001f
FLASH	0x00040	0x0103f
RAM	0x10000	0x1ffff

DA VERIFICARE In the `MEMORY` example the RAM block has `origin = 0x1000h`, `length = 0x10000h`, while the memory map on the slide shows RAM from `0x10000` to `0x1ffff`. The mismatch between code and map is in the slide.

SECTION Directive

- The `SECTION` directive tells the linker:
 - which input sections are to be combined into which output section;
 - which output sections are to be grouped together and allocated in contiguous memory;
 - where to place each section.

```
SECTION {
    output-section-name : { contents } > area-name
    ...
    GROUP {
        [ALIGN(expression)]
        section-definition
        ...
    } > area-name
}
```

Example: two object files generated by a compiler or assembler (`file1.o` and `file2.o`); three default sections (`.text`, `.data`, `.bss`); two developer-specified sections (`loader` and `my_section`).

```
SECTION {
    .text :
    {
        my_section
        *(.text)
    }
    loader : > FLASH
    GROUP ALIGN (4) :
    {
        .data : {}
        .bss : {}
    } >RAM
}
```

Input section	Source	Output section
loader	file1.o	loader section → FLASH
.text	file1.o, file2.o	.text section
my_section	file2.o	.text section
.data	file1.o, file2.o	.data section
.bss	file1.o, file2.o	.bss section

Fig. 4: Sections from `file1.o` and `file2.o` merged into the executable image

- Combine `my_section` and the `.text` sections from all object files into the final output `.text` section.
- The `loader` section is placed into flash memory.
- `.data` and `.bss` are grouped and allocated in contiguous physical RAM, aligned on the 4-byte boundary.